

Docker Image Hands on

1. Check if docker is installed or not, on lab machine its already installed but if you are installing docker on another VM or machine, need to install it separately from <https://docs.docker.com/engine/install/>.

```
(base) labuser@ip-172-31-27-175:~$ docker --version
Docker version 27.1.1, build 6312585
```

2. Check if you have any docker image on the lab/local machine. If not create one using Dockerfile.

```
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
docker-example      latest         795e39f6776b   2 minutes ago   55MB
```

3. Tag the image with a custom tag (different from default LATEST tag).

`docker tag docker-example docker-example:v0.1`

So same image is now replicated with 2 different tags “latest” & “v0.1”. (confirmed via same IMAGE ID).

```
(base) labuser@ip-172-31-27-175:~$ docker tag docker-example docker-example:v0.1
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
docker-example      latest         795e39f6776b   2 minutes ago   55MB
docker-example      v0.1           795e39f6776b   2 minutes ago   55MB
(base) labuser@ip-172-31-27-175:~$
```

4. Make any small change in Dockerfile, and rebuild the image :

```
# 1. Base image
FROM nginx:alpine
# 2. Metadata about the image
LABEL description="Nginx web server serving static HTML and Docker"
# 4. Build-time temp variable (can be overridden at build time), this is not persisting inside container
ARG PAGE_TITLE="Welcome to Dockerized Nginx V0.2"
# 5. Set working directory, container will start with this Home directory
WORKDIR /usr/share/nginx/html
# 6. Copy custom HTML file to web root
COPY index.html /usr/share/nginx/html/
# 3. Set environment variable. Can be override during docker run.
ENV KUBECTL_VERSION=v1.29.0
# 7. ADD can copy additional Remote URL and auto extract .tar file. Like kubectl
ADD https://github.com/jqlang/jq/releases/download/jq-1.8.1/jq-1.8.1.tar.gz /tmp/
# 8. Declare a volume for persistent/static content bound to host machine (optional)
VOLUME ["/usr/share/nginx/html"]
# 9. Run a command during build
RUN mkdir -p /var/cache/nginx/client_temp /run && \
    chown -R nginx:nginx /var/cache/nginx /run
#10. Set a custom user (nginx runs as this user by default, can be checked with "user" command in container.)
USER root
# 11. Document exposed port (used by Nginx)
EXPOSE 80
# 12. Use custom entrypoint (overrides default, optional)
ENTRYPOINT ["/bin/sh", "-c"]
# 13. Default command (print a welcome message and start nginx)
```

`docker build -t docker-example:latest .`

Note: It's a good practice to use “latest” tag in build command and later tag it with custom value, this ensures that any application, which is using this image, will pick the “latest” tag which is assigned to the image built with latest code changes.

5. A new image will be visible with different IMAGE ID.

```
=> exporting to image
=> => exporting layers
=> => writing image sha256:f64ac14fdd0dabcf3285fadde65962d5cf857471e5fe08d89e188d37b1d21eec
=> => naming to docker.io/library/docker-example:latest
(base) labuser@ip-172-31-27-175:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
docker-example	latest	f64ac14fdd0d	37 seconds ago	55MB
docker-example	v0.1	795e39f6776b	3 minutes ago	55MB

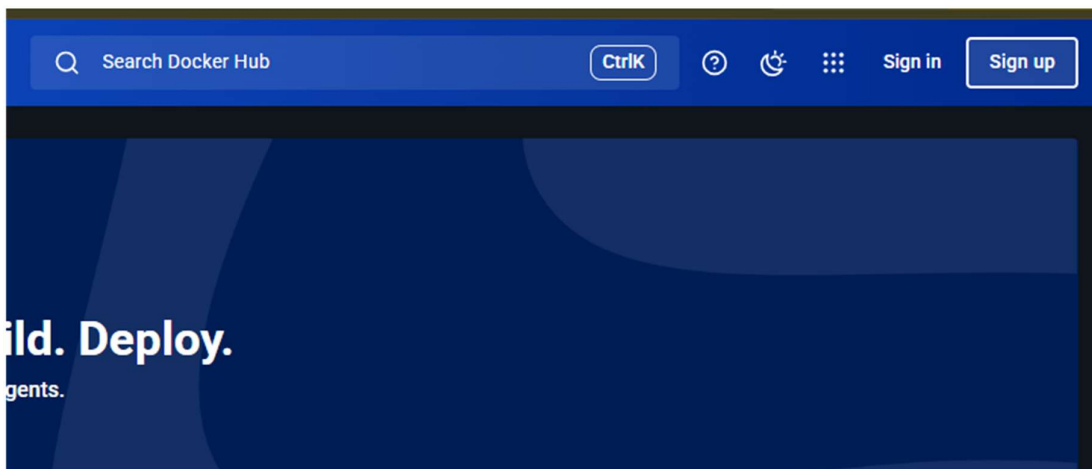
We can now tag it an additional tag like v0.2 or something. Tagging is like creating a different copy of same image but with a different name/tag.

```
(base) labuser@ip-172-31-27-175:~$ docker tag docker-example docker-example:v0.2
(base) labuser@ip-172-31-27-175:~$ docker images
```

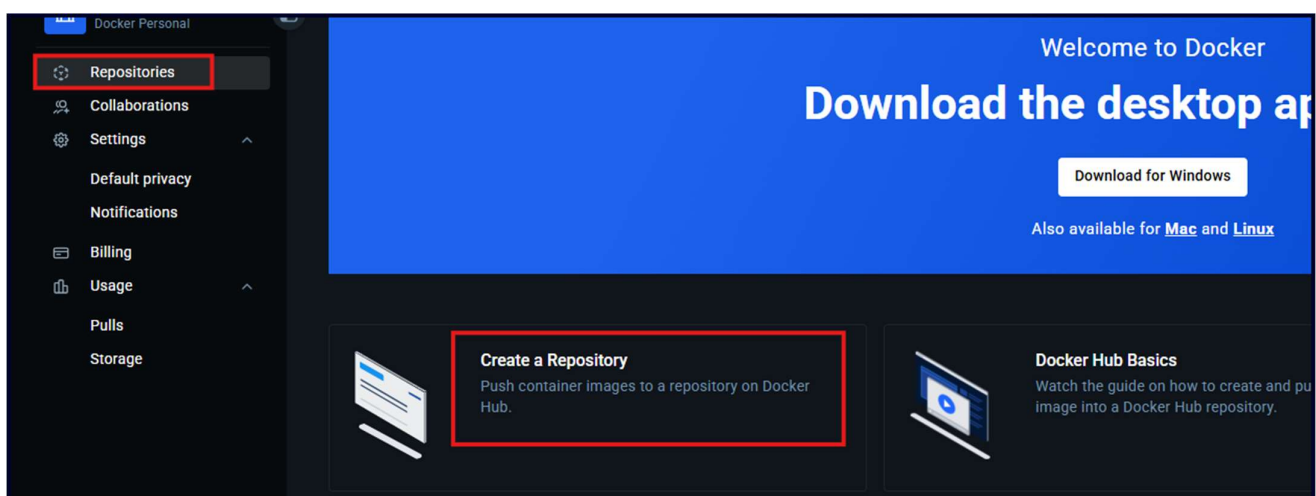
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
docker-example	latest	f64ac14fdd0d	About a minute ago	55MB
docker-example	v0.2	f64ac14fdd0d	About a minute ago	55MB
docker-example	v0.1	795e39f6776b	4 minutes ago	55MB

So if we now create a container with image “**docker-example:latest**” it will use the image which we recently created after a dummy change (IMAGE ID f64ac14fdd0d) and not the very first image which we had in our machine.

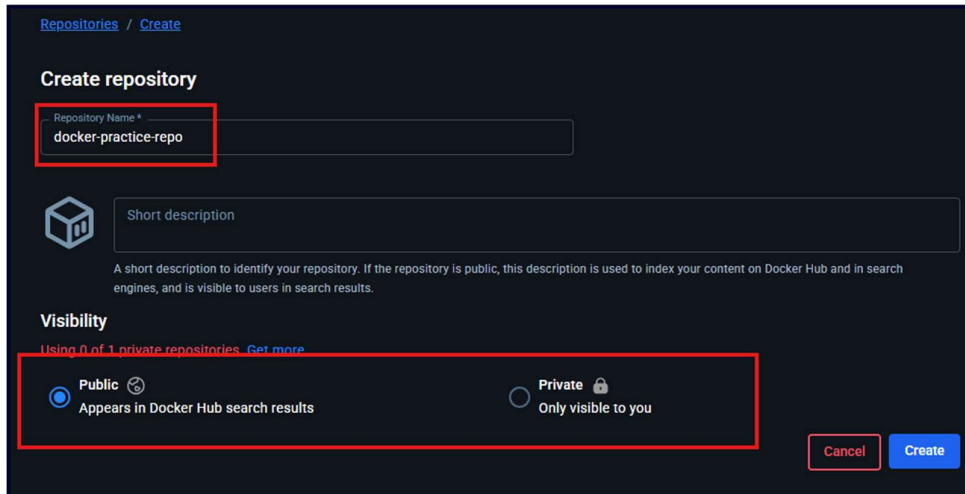
6. Now its time to upload our images to Docker Hub @ <https://hub.docker.com/>
Click On Signup and you can use Google or GitHub account for a quick signup.



7. On the left hand side, click on Repositories



8. Provide a repository name and choose Public (so that anyone can access without credentials or Access token). In Enterprises , we usually have private repositories which will require access based control over these repositories.



Repositories / Create

Create repository

Repository Name *

docker-practice-repo

Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Get more](#)

☒ Public  Appears in Docker Hub search results

☐ Private  Only visible to you

Cancel Create

9. We need to authenticate with docker before we can push the image. Pull can be done without login if image belongs to public repository like ours.

```
(base) labuser@ip-172-31-27-175:~$ docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to get one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations.

Username: saurabhnegi2306
Password:
WARNING! Your password will be stored unencrypted in /home/labuser/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

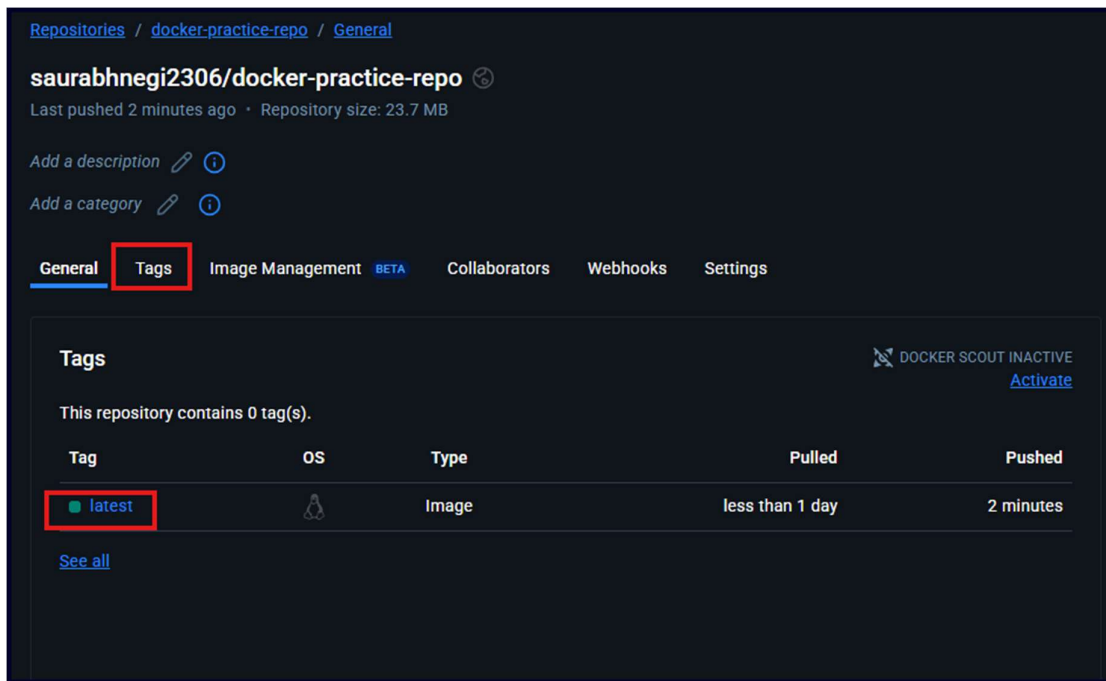
Login Succeeded
(base) labuser@ip-172-31-27-175:~$
```

10. Before we can push the image to our public repository, we need to tag it properly with the `<<DOCKERHUB-ACCOCUNT>>/<<repository-name>>:<tag-name>`

```
docker tag docker-example:latest saurabhnegi2306/docker-practice-repo:latest
docker push saurabhnegi2306/docker-practice-repo:latest
```

```
(base) labuser@ip-172-31-27-175:~$ docker tag docker-example:latest saurabhnegi2306/docker-practice-repo:latest
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
saurabhnegi2306/docker-practice-repo     latest   f64ac14fdd0d  39 minutes ago  55MB
docker-example                           latest   f64ac14fdd0d  39 minutes ago  55MB
docker-example                           v0.2     f64ac14fdd0d  39 minutes ago  55MB
docker-example                           v0.1     795e39f6776b  42 minutes ago  55MB
(base) labuser@ip-172-31-27-175:~$ docker push saurabhnegi2306/docker-practice-repo:latest
The push refers to repository [docker.io/saurabhnegi2306/docker-practice-repo]
5a1e1d3567e7: Pushed
2933c7dbb744: Pushed
114de00dfe56: Pushed
5f70bf18a086: Pushed
8edc3fb8d5af: Pushed
db8f95327d8f: Layer already exists
c52d5b69a95e: Layer already exists
f214d6d47d3f: Layer already exists
a31784bb04f7: Layer already exists
2a18c8350ab6: Layer already exists
bd37660aadab: Pushed
0b83d017db6e: Pushed
latest: digest: sha256:384a015337ada43678f1270b683a7c204280c1708d9c5fb65d77b7795699850f size: 2820
(base) labuser@ip-172-31-27-175:~$
```

11. Open the docker hub repository page, a new image should be available now. Click on tag name to see which instructions are used to create this image.



The screenshot shows the Docker Hub repository page for `saurabhnegi2306/docker-practice-repo`. The page is in the 'Tags' tab, which is highlighted with a red box. The repository has a size of 23.7 MB and was last pushed 2 minutes ago. The 'Tags' section shows a table with one tag, 'latest', which is also highlighted with a red box. The table columns are Tag, OS, Type, Pulled, and Pushed. The 'latest' tag is of type 'Image' and was pushed less than 1 day ago, 2 minutes ago. There is a 'See all' link below the table. A 'DOCKER SCOUT INACTIVE' banner is visible in the top right corner of the repository page.

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	2 minutes

12. This image is now publicly available on the docker hub and anyone can pull this image without docker login and launch a container from this image :

```
docker run -p 8081:80 saurabhnegi2306/docker-practice-repo:latest OR  
docker run -d -p 8081:80 saurabhnegi2306/docker-practice-repo:latest
```


AWS Elastic Container Registry Hands on

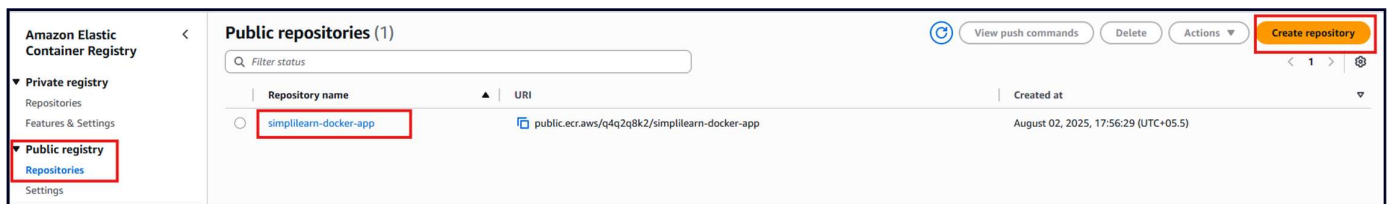
If you have access to an AWS account , you can use AWS ECR to store and manage docker images just like Dockerhub.

1. Use AWS Cli to configure AWS credentials on your lab machine using “aws configure” :

```
(base) labuser@ip-172-31-27-175:~$ aws configure
AWS Access Key ID [*****QXNS]: AKIAIYZ2R2TMBV37QXNS
AWS Secret Access Key [*****cqmb]: 9FA611CIImoGgRJw6Uxx2UVs0x1Ig0nzZxBQUcqmb
Default region name [us-east-1]: us-east-1
Default output format [json]: json
(base) labuser@ip-172-31-27-175:~$
(base) labuser@ip-172-31-27-175:~$ aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/q4q2q8k2
WARNING! Your password will be stored unencrypted in /home/labuser/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

Login Succeeded
(base) labuser@ip-172-31-27-175:~$
```

2. Open AWS ECR console, Create a Public Repository just for exercise.



3. Tag an existing image in AWS ECR format (URI in ECR console):

`docker tag docker-example public.ecr.aws/q4q2q8k2/simplilearn-docker-app:latest`

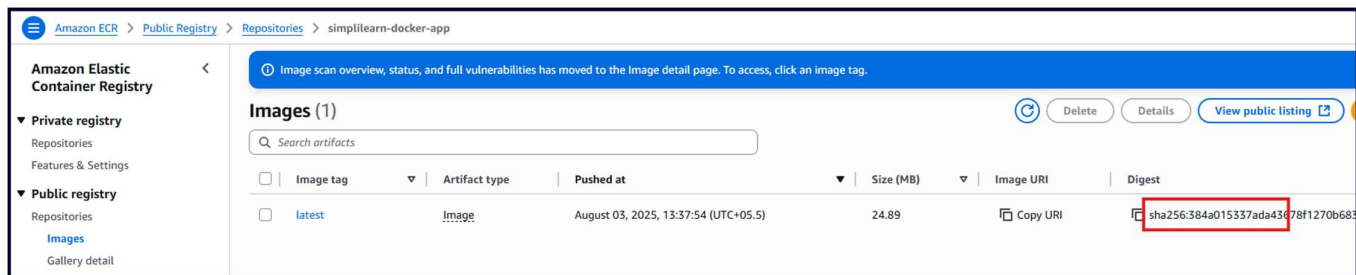
4. Push the newly tagged image to ECR using :

`docker push public.ecr.aws/q4q2q8k2/simplilearn-docker-app:latest`

```
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
docker-example      latest       f64ac14fdd0d     48 minutes ago  55MB
docker-example      v0.2        f64ac14fdd0d     48 minutes ago  55MB
saunabhnegi2306/docker-practice-repo latest       f64ac14fdd0d     48 minutes ago  55MB
docker-example      v0.1        795e39f6776b     51 minutes ago  55MB
(base) labuser@ip-172-31-27-175:~$ docker tag docker-example public.ecr.aws/q4q2q8k2/simplilearn-docker-app:latest
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
saunabhnegi2306/docker-practice-repo latest       f64ac14fdd0d     49 minutes ago  55MB
public.ecr.aws/q4q2q8k2/simplilearn-docker-app latest       f64ac14fdd0d     49 minutes ago  55MB
docker-example      latest       f64ac14fdd0d     49 minutes ago  55MB
docker-example      v0.2        f64ac14fdd0d     49 minutes ago  55MB
docker-example      v0.1        795e39f6776b     51 minutes ago  55MB
(base) labuser@ip-172-31-27-175:~$ docker push public.ecr.aws/q4q2q8k2/simplilearn-docker-app:latest

The push refers to repository [public.ecr.aws/q4q2q8k2/simplilearn-docker-app]
5a1e1d3567e7: Pushed
2933c7dbb744: Pushed
114de00dfe56: Pushed
5f70bf18a086: Pushed
8edc3fb8d5af: Pushed
db8f95327d8f: Pushed
c52d5b69a95e: Pushed
f214d6d47d3f: Pushed
a31784bb04f7: Pushed
2a18c8350ab6: Pushed
bd37660aadab: Pushed
0b83d017db6e: Pushed
latest: digest: sha256:384a015337ada43678f1276b683a7c204280c1708d9c5fb65d77b7795699850f size: 2820
(base) labuser@ip-172-31-27-175:~$
```

5. We should now have the image available in our AWS ECR repository :



6. Test the connectivity by launching a container with this image :

```
docker run -p 8081:80 public.ecr.aws/q4q2q8k2/simplilearn-docker-app:latest
```

OR

```
docker run -d -p 8081:80 public.ecr.aws/q4q2q8k2/simplilearn-docker-app:latest  
( Detached mode)
```